Go Goroutines

The Complete Reference

Concurrency · Channels · Synchronization · Patterns · Best Practices

---

Comprehensive Technical Guide • All Versions of Go

1. Introduction to Goroutines

A goroutine is a lightweight, independently executing function managed entirely by the Go runtime. Unlike OS threads, which are managed by the kernel, goroutines are multiplexed onto a smaller number of OS threads by the runtime scheduler. This design makes it practical to spawn hundreds of thousands — or even millions — of goroutines in a single process without exhausting memory or hitting OS limits.

Goroutines are the primary unit of concurrency in Go. The language and runtime were designed from the ground up around concurrency, drawing on the theoretical model of Communicating Sequential Processes (CSP) by Tony Hoare (1978). The core philosophy is: do not communicate by sharing memory; instead, share memory by communicating.

1.1 Goroutines vs OS Threads

Property	Goroutine	OS Thread
Initial stack size	2–8 KB (grows/shrinks dynamically)	1–8 MB (fixed at creation)
Creation cost	Microseconds, a few KB	Milliseconds, megabytes
Scheduling	Go runtime (cooperative + preemptive, user space)	OS kernel (preemptive)
Communication	Channels (CSP model)	Shared memory + locks
Multiplexing	M goroutines on N OS threads (M:N)	1 thread per kernel thread (1:1)
Context switch	Very cheap (user space)	Expensive (kernel trap + context save)
Max practical count	Millions in a single process	Thousands (OS limit)

1.2 Launching a Goroutine

Any function call prefixed with the go keyword launches that function as a new goroutine. The go statement returns immediately; the goroutine runs concurrently alongside the caller.

```
package main

import (
    "fmt"
    "time"
)

func sayHello(name string) {
    fmt.Printf("Hello, %s!\n", name)
```

```
}

func main() {
    go sayHello("Alice")  // launch goroutine — returns immediately
    go sayHello("Bob")    // launch another goroutine

    // main must wait; when main returns all goroutines are killed
    time.Sleep(100 * time.Millisecond)
    fmt.Println("main exits")
}
```

Warning: When main() returns, every goroutine is killed immediately — even ones still running. Never rely on time.Sleep to wait for goroutines in production. Use sync.WaitGroup or channels instead.

1.3 Anonymous Goroutines (Closures)

Goroutines are most commonly launched as anonymous functions (closures). A closure captures variables from its surrounding scope. Be careful: capturing a loop variable by reference is a frequent source of bugs.

```
func main() {
    msg := "hello"
    go func() {
        fmt.Println(msg) // captures msg from enclosing scope
    }()

    // --- Loop variable capture bug ---
    // WRONG: all goroutines may print the same final value of i
    for i := 0; i < 3; i++ {
        go func() { fmt.Println(i) }()  // i is shared, NOT copied
    }

    // CORRECT: pass i as an argument so each goroutine gets its own copy
    for i := 0; i < 3; i++ {
        go func(n int) { fmt.Println(n) }(i)
    }
    time.Sleep(time.Second)
}
```

Go 1.22+ Fix: Starting with Go 1.22, loop variables are re-scoped per iteration, so the classic loop-capture bug no longer applies to for-range loops. But explicit argument passing is still clearer and compatible with all versions.

2. The Go Runtime Scheduler

The Go runtime implements an M:N scheduler — it multiplexes M goroutines across N OS threads using a cooperative-and-preemptive hybrid approach. Understanding the scheduler helps you write efficient concurrent programs and diagnose subtle performance issues.

2.1 The GMP Model

The scheduler revolves around three core entities:

Entity	Full Name	Role
G	Goroutine	Represents a goroutine: its stack, registers, and state (runnable, running, waiting, dead).
M	Machine (OS thread)	An OS thread. An M executes G code. The runtime creates and reuses Ms.
P	Processor (logical)	A logical processor that mediates between Ms and Gs. Holds a local run queue (LRQ) of runnable Gs. The number of Ps equals GOMAXPROCS.

Each P maintains a Local Run Queue (LRQ). There is also a Global Run Queue (GRQ) for goroutines without a P. When a P's LRQ is empty, it performs work stealing — taking goroutines from the LRQ of another P or from the GRQ.

2.2 GOMAXPROCS

GOMAXPROCS controls how many Ps (and thus how many goroutines) can run simultaneously. It defaults to the number of logical CPU cores. Increasing it allows more parallelism for CPU-bound work; decreasing it can be useful in containerized environments.

```
import "runtime"

// Query current value (0 = query without changing)
fmt.Println(runtime.GOMAXPROCS(0))

// Set explicitly
runtime.GOMAXPROCS(4)

// Also set via environment variable before running the program:
// GOMAXPROCS=4 go run main.go
```

2.3 Goroutine States

State	Description
Runnable (_Grunnable)	Ready to execute. Sitting in a run queue, waiting for a free P.
Running (_Grunning)	Currently executing on an OS thread (M). At most one goroutine per P.
Waiting (_Gwaiting)	Blocked — waiting on a channel operation, syscall, timer, sync.Mutex, GC, or network I/O.
Dead (_Gdead)	Has finished execution or was newly allocated (recycled from dead pool).
Syscall (_Gsyscall)	Inside a blocking syscall. The P is released so other goroutines can run.

2.4 Preemption (Go 1.14+)

Before Go 1.14, goroutines could only be preempted at function call boundaries. A CPU-bound goroutine with no function calls could starve the scheduler. Since Go 1.14, the runtime uses asynchronous preemption: it sends a SIGURG signal (Unix) or uses Structured Exception Handling (Windows) to interrupt a goroutine at almost any safe instruction point. This means CPU-bound goroutines no longer block others from running.

2.5 Syscalls and Network Poller

When a goroutine makes a blocking syscall (e.g., file I/O), the runtime detaches the P from the M (so the P can run other goroutines on a new M) and parks the goroutine until the syscall returns. For non-blocking network I/O, the runtime uses a platform-native poller (epoll on Linux, kqueue on macOS) — goroutines doing network I/O are parked and rescheduled automatically when data arrives, achieving high I/O concurrency without blocking OS threads.

3. Channels

A channel is a typed conduit for values. It allows goroutines to communicate and synchronize without sharing memory. Channels are first-class values in Go — they can be stored in variables, passed to functions, and returned from functions.

3.1 Creating Channels

```
ch  := make(chan int)        // unbuffered channel of int
bch := make(chan string, 10) // buffered channel, capacity 10

// Typed — a channel only carries its declared type
type Job struct{ ID int }
jobs := make(chan Job, 100)

// Directional channel variables
var recv <-chan int = ch  // receive-only
var send  chan<- int = ch // send-only
```

3.2 Send, Receive, and the ok Idiom

```
ch := make(chan int, 1)

// Send
ch <- 42

// Receive
val := <-ch
fmt.Println(val) // 42

// Receive with closed-channel check
val, ok := <-ch
if !ok {
    fmt.Println("channel is closed and drained")
}
```

3.3 Unbuffered vs Buffered Channels

Property	Unbuffered (cap 0)	Buffered (cap N)
Send blocks when	No receiver is ready	Buffer is full
Receive blocks when	No sender is ready	Buffer is empty
Synchronization	Strong — both goroutines rendezvous at the channel operation	Weak — sender and receiver are decoupled up to N values

Typical use	Signaling, handshake, pipeline stages	Burst buffering, work queues, smoothing producer-consumer speed differences

```
// Unbuffered: send and receive must happen simultaneously
ch := make(chan int)
go func() { ch <- 99 }()
fmt.Println(<-ch) // 99

// Buffered: sender proceeds without a waiting receiver (up to cap)
bch := make(chan int, 3)
bch <- 1; bch <- 2; bch <- 3 // all succeed immediately
// bch <- 4  // would block — buffer full
fmt.Println(<-bch, <-bch, <-bch) // 1 2 3
```

3.4 Closing Channels

Closing a channel signals that no more values will be sent. The receiver can detect this and drain remaining values. Ranging over a channel exits automatically when the channel is closed and empty.

```
ch := make(chan int, 5)
for i := 0; i < 5; i++ { ch <- i }
close(ch) // signal: no more values

for v := range ch { // exits when ch is closed and empty
    fmt.Println(v) // 0 1 2 3 4
}
```

Channel Close Rules: (1) Only the SENDER closes a channel. (2) Sending on a closed channel panics. (3) Receiving from a closed channel returns the zero value immediately with ok=false. (4) Closing an already-closed channel panics.

3.5 The select Statement

select is like a switch for channel operations. It blocks until at least one case can proceed, then picks one at random if multiple are ready. select is the primary tool for multiplexing channels, implementing timeouts, and non-blocking channel operations.

```
ch1 := make(chan string, 1)
ch2 := make(chan string, 1)

go func() { time.Sleep(100*time.Millisecond); ch1 <- "one" }()
go func() { time.Sleep(200*time.Millisecond); ch2 <- "two" }()
```

```
// Wait for whichever comes first
for i := 0; i < 2; i++ {
    select {
    case msg := <-ch1:
        fmt.Println("ch1:", msg)
    case msg := <-ch2:
        fmt.Println("ch2:", msg)
    }
}
```

Non-blocking operations with default

```
select {
case v := <-ch:
    fmt.Println("received:", v)
default:
    fmt.Println("nothing ready")
}
```

Timeout with time.After

```
select {
case result := <-workCh:
    fmt.Println("got result:", result)
case <-time.After(5 * time.Second):
    fmt.Println("timed out after 5s")
}
```

Draining a channel on shutdown

```
done := make(chan struct{})
// ... launch goroutines that send to workCh

close(done) // signal all workers

// Drain any remaining results
for {
    select {
    case result := <-resultCh:
        process(result)
    default:
        return // nothing left
    }
}
```

3.6 Channel Directions in Function Signatures

Constraining channel direction in function signatures makes intent clear and prevents accidental misuse at compile time:

```
func produce(out chan<- int, n int) { // send-only: cannot receive
    for i := 0; i < n; i++ { out <- i }
    close(out)
}

func consume(in <-chan int) { // receive-only: cannot send or close
    for v := range in { fmt.Println(v) }
}

func main() {
    ch := make(chan int, 10)
    go produce(ch, 5) // bidirectional ch converts to chan<-int
    consume(ch)        // bidirectional ch converts to <-chan int
}
```

3.7 nil Channels

A nil channel (var ch chan int) blocks forever on both send and receive. This is useful in select statements to disable a case dynamically:

```
var ch chan int // nil channel

// In a select, a nil channel case is never selected:
select {
case v := <-ch:    // never fires because ch == nil
    fmt.Println(v)
case <-time.After(time.Second):
    fmt.Println("timeout")
}
```

4. Synchronization Primitives (sync Package)

The sync package provides lower-level synchronization primitives for protecting shared state. While channels are idiomatic for goroutine communication, mutexes and atomics are preferred when many goroutines access the same memory location without goroutine-to-goroutine handoff.

4.1 sync.WaitGroup

WaitGroup waits for a collection of goroutines to finish. The main goroutine calls Add to set the count, each goroutine calls Done when finished, and Wait blocks until the counter reaches zero.

```
var wg sync.WaitGroup

for i := 0; i < 5; i++ {
    wg.Add(1)            // add BEFORE launching goroutine
    go func(n int) {
        defer wg.Done() // always defer — ensures Done even on panic
        fmt.Printf("worker %d done\n", n)
    }(i)
}

wg.Wait() // blocks until all Done() calls balance the Add() calls
fmt.Println("all done")
```

Critical: Always call wg.Add() BEFORE launching the goroutine, not inside it. If the scheduler runs before Add(), Wait() may return prematurely.

4.2 sync.Mutex

Mutex (mutual exclusion lock) ensures only one goroutine executes a critical section at a time. Use it to guard shared mutable state.

```
type SafeMap struct {
    mu sync.Mutex
    m  map[string]int
}

func (s *SafeMap) Set(key string, val int) {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.m[key] = val
}

func (s *SafeMap) Get(key string) int {
    s.mu.Lock()
```

```
    defer s.mu.Unlock()
    return s.m[key]
}
```

Best Practice: Never copy a sync.Mutex by value — it contains internal state. Always embed it or use a pointer. go vet will warn about copying sync types.

4.3 sync.RWMutex

RWMutex allows multiple concurrent readers OR one exclusive writer. Use it when reads significantly outnumber writes, as it allows parallel read throughput.

```
var (
    rw    sync.RWMutex
    store = map[string]int{}
)

func read(key string) int {
    rw.RLock()           // many goroutines can hold RLock simultaneously
    defer rw.RUnlock()
    return store[key]
}

func write(key string, val int) {
    rw.Lock()            // exclusive — blocks all readers and writers
    defer rw.Unlock()
    store[key] = val
}
```

4.4 sync.Once

Once ensures a function runs exactly once, no matter how many goroutines call Do concurrently. All callers block until the first invocation completes. Ideal for lazy, goroutine-safe initialization.

```
var (
    instance *DB
    once     sync.Once
)

func GetDB() *DB {
    once.Do(func() {
        instance = connectToDB() // called exactly once
    })
    return instance
}
```

4.5 sync.Cond

Cond implements a condition variable. One or more goroutines wait for a condition to become true while another goroutine signals it. Less common than channels but useful when the condition depends on complex shared state.

```
var mu sync.Mutex
cond := sync.NewCond(&mu)
ready := false

// Waiter goroutine
go func() {
    mu.Lock()
    for !ready {       // always loop — spurious wakeups can occur
        cond.Wait()    // atomically: unlock mu, suspend, re-lock on wake
    }
    fmt.Println("condition met")
    mu.Unlock()
}()

// Signaler
time.Sleep(500 * time.Millisecond)
mu.Lock()
ready = true
cond.Signal()    // wake one waiting goroutine
// cond.Broadcast() // wake ALL waiting goroutines
mu.Unlock()
```

4.6 sync.Map

sync.Map is a goroutine-safe map optimized for two patterns: a key-value pair written once and read many times, or keys updated by disjoint goroutines. It trades type safety (uses interface{}/any) for built-in lock-free reads.

```
var m sync.Map

m.Store("go", 1)                          // write
val, ok := m.Load("go")                   // read
actual, loaded := m.LoadOrStore("go", 99) // load existing or store new
m.Delete("go")                            // delete

m.Range(func(key, val any) bool {  // iterate — no guaranteed order
    fmt.Println(key, val)
    return true // return false to stop early
})
```

4.7 sync/atomic Package

The sync/atomic package provides low-level atomic memory operations that are faster than mutexes for simple integer counters and flag variables.

```
import "sync/atomic"

var counter int64

atomic.AddInt64(&counter, 1)           // increment
atomic.AddInt64(&counter, -1)          // decrement
n := atomic.LoadInt64(&counter)        // read
atomic.StoreInt64(&counter, 0)         // write
old := atomic.SwapInt64(&counter, 100) // swap, returns old value

// Compare-and-swap: set to newVal only if current == oldVal
swapped := atomic.CompareAndSwapInt64(&counter, 100, 200)

// atomic.Value: store/load any type atomically
var v atomic.Value
v.Store([]string{"a", "b"})             // store pointer-sized value
arr := v.Load().([]string)             // load and type-assert
```

5. The context Package

The context package provides a standard mechanism for carrying cancellation signals, deadlines, and request-scoped values across API boundaries and through chains of goroutines. It is the idiomatic way to stop goroutines in Go services.

5.1 Context Constructors

Function	Description
context.Background()	The root context. Never cancelled, no deadline, no values. The default starting point for all contexts.
context.TODO()	A placeholder context for code under construction. Signals that the right context has not been determined yet.
context.WithCancel(parent)	Returns a new context and a cancel function. Calling cancel() cancels this context and all its children.
context.WithTimeout(parent, d)	Cancelled automatically after duration d, or when cancel() is called — whichever comes first.
context.WithDeadline(parent, t)	Cancelled at absolute time t, or when cancel() is called.
context.WithValue(parent, k, v)	Returns a copy of parent carrying a key-value pair. Use ONLY for request-scoped data (trace IDs, auth).

5.2 Cancelling a Single Goroutine

```
func worker(ctx context.Context) {
    for {
        select {
        case <-ctx.Done():
            fmt.Println("worker stopped:", ctx.Err())
            return
        default:
            doUnitOfWork()
        }
    }
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
    defer cancel() // always defer cancel to free resources
    go worker(ctx)
    time.Sleep(3 * time.Second)
}
```

5.3 Cancelling Multiple Goroutines

Cancelling a parent context automatically cancels all child contexts derived from it. This is the standard way to shut down a group of goroutines simultaneously.

```
func main() {
    ctx, cancel := context.WithCancel(context.Background())
    var wg sync.WaitGroup

    for i := 0; i < 5; i++ {
        wg.Add(1)
        go func(id int, ctx context.Context) {
            defer wg.Done()
            for {
                select {
                case <-ctx.Done():
                    fmt.Printf("goroutine %d exiting\n", id)
                    return
                default:
                    time.Sleep(100 * time.Millisecond)
                }
            }
        }(i, ctx)
    }

    time.Sleep(500 * time.Millisecond)
    cancel() // cancels all 5 goroutines simultaneously
    wg.Wait()
}
```

5.4 Propagating Context Through Call Chains

```
// Context should always be the first parameter
func queryDB(ctx context.Context, id int) (*User, error) {
    // Pass ctx down to sub-operations
    return db.QueryRowContext(ctx, "SELECT * FROM users WHERE id=?", id)
}

func handleRequest(ctx context.Context) {
    user, err := queryDB(ctx, 42)
    // ...
}
```

5.5 context.WithValue

```
// Use unexported type as key to avoid collisions
type ctxKey string
const requestIDKey ctxKey = "requestID"

func withRequestID(ctx context.Context, id string) context.Context {
    return context.WithValue(ctx, requestIDKey, id)
}
```

```
func getRequestID(ctx context.Context) (string, bool) {
    id, ok := ctx.Value(requestIDKey).(string)
    return id, ok
}
```

Warning: context.WithValue is NOT a general-purpose parameter passing mechanism. Use it only for request-scoped data that crosses API or process boundaries (e.g., trace IDs, auth tokens). Never use it for optional function parameters.

6. Common Concurrency Patterns

6.1 Pipeline

A pipeline is a series of stages where each stage receives values from an upstream channel, processes them, and sends results to a downstream channel. Pipelines compose naturally and enable clean, testable stream processing.

```
func generate(nums ...int) <-chan int {
    out := make(chan int)
    go func() { defer close(out); for _, n := range nums { out <- n } }()
    return out
}

func square(in <-chan int) <-chan int {
    out := make(chan int)
    go func() { defer close(out); for n := range in { out <- n * n } }()
    return out
}

func main() {
    for v := range square(square(generate(2, 3, 4))) {
        fmt.Println(v) // 16, 81, 256
    }
}
```

6.2 Fan-Out / Fan-In

Fan-out distributes work from one channel to multiple goroutines. Fan-in merges results from multiple channels into one. Together they implement parallel processing with result aggregation.

```
// Fan-out: start N workers all reading from the same input channel
func fanOut(in <-chan int, n int) []<-chan int {
    outs := make([]<-chan int, n)
    for i := range outs { outs[i] = worker(in) }
    return outs
}

// Fan-in: merge multiple channels into one
func fanIn(channels ...<-chan int) <-chan int {
    merged := make(chan int)
    var wg sync.WaitGroup
    forward := func(ch <-chan int) {
        defer wg.Done()
        for v := range ch { merged <- v }
    }
    wg.Add(len(channels))
    for _, ch := range channels { go forward(ch) }
    go func() { wg.Wait(); close(merged) }()
    return merged
```

```
}
```

6.3 Worker Pool

A worker pool limits concurrency to a fixed number of goroutines. N workers drain a jobs channel, preventing goroutine explosion under high load.

```
func runPool(numWorkers, numJobs int) {
    jobs    := make(chan int, numJobs)
    results := make(chan int, numJobs)

    // Launch fixed number of workers
    for w := 0; w < numWorkers; w++ {
        go func() {
            for j := range jobs {
                results <- process(j)
            }
        }()
    }

    // Send all jobs then close to signal no more work
    for j := 0; j < numJobs; j++ { jobs <- j }
    close(jobs)

    // Collect results
    for r := 0; r < numJobs; r++ { fmt.Println(<-results) }
}
```

6.4 Semaphore (Bounded Concurrency)

A buffered channel acts as a counting semaphore: send to acquire, receive to release. This limits how many goroutines enter a resource-intensive section at once.

```
sem := make(chan struct{}, 5) // allow at most 5 concurrent goroutines

var wg sync.WaitGroup
for i := 0; i < 100; i++ {
    wg.Add(1)
    go func(n int) {
        defer wg.Done()
        sem <- struct{}{}  // acquire
        defer func() { <-sem }() // release
        expensiveWork(n)
    }(i)
}
wg.Wait()
```

6.5 Done Channel (Manual Cancellation)

Before context was added (Go 1.7), the done channel pattern was the standard way to cancel goroutines. It remains useful for understanding context's internals.

```
func producer(done <-chan struct{}) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for i := 0; ; i++ {
            select {
            case out <- i:
            case <-done:
                return
            }
        }
    }()
    return out
}

func main() {
    done := make(chan struct{})
    nums := producer(done)
    for i := 0; i < 5; i++ { fmt.Println(<-nums) }
    close(done) // signal producer to stop
}
```

6.6 Heartbeat Pattern

A goroutine sends periodic heartbeat signals so the caller can detect if it is still alive or has stalled:

```
func worker(done <-chan struct{}, heartbeat time.Duration) (<-chan int, <-
chan time.Time) {
    results   := make(chan int)
    heartbeats := make(chan time.Time)
    go func() {
        defer close(results)
        tick := time.NewTicker(heartbeat)
        defer tick.Stop()
        for i := 0; ; i++ {
            select {
            case <-done: return
            case heartbeats <- tick.C: // non-blocking
            default:
            }
            select {
            case <-done: return
            case results <- i:
            }
        }
    }()
```

```
    return results, heartbeats
}
```

6.7 Or-Done Channel

Wrap a channel so that receiving from it also respects a done/cancel channel, without modifying the underlying channel:

```
func orDone(done, ch <-chan interface{}) <-chan interface{} {
    relayStream := make(chan interface{})
    go func() {
        defer close(relayStream)
        for {
            select {
            case <-done: return
            case v, ok := <-ch:
                if !ok { return }
                select {
                case relayStream <- v:
                case <-done: return
                }
            }
        }
    }()
    return relayStream
}
```

6.8 errgroup (golang.org/x/sync)

errgroup is a higher-level abstraction over WaitGroup that also captures and returns the first non-nil error from any goroutine in the group. It is the idiomatic way to run a group of goroutines where any one can fail.

```
import "golang.org/x/sync/errgroup"

func fetchAll(ctx context.Context, urls []string) error {
    g, ctx := errgroup.WithContext(ctx)
    for _, u := range urls {
        u := u // capture loop variable
        g.Go(func() error {
            resp, err := http.Get(u)
            if err != nil { return err }
            defer resp.Body.Close()
            fmt.Println(u, resp.Status)
            return nil
        })
    }
    return g.Wait() // returns first non-nil error
}
```

7. Data Races and the Race Detector

A data race occurs when two or more goroutines access the same memory location concurrently, at least one access is a write, and the accesses are not protected by a synchronization mechanism. Data races are undefined behavior in Go: they can cause crashes, corrupt data, or produce silently incorrect results.

7.1 Classic Race Condition

```
// DATA RACE — do not use
var counter int

func main() {
    var wg sync.WaitGroup
    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            counter++ // read-modify-write is NOT atomic
        }()
    }
    wg.Wait()
    fmt.Println(counter) // could be anything from 1 to 1000
}
```

7.2 Enabling the Race Detector

```
go run  -race main.go          # run with race detection
go test -race ./...            # test entire project
go build -race -o app .        # build a race-instrumented binary
go test -race -count=20 ./...  # run tests 20x to catch intermittent races
```

The race detector instruments every memory access at compile time and reports races at runtime with full stack traces showing the conflicting goroutines. It incurs roughly 5-10x CPU overhead and 5-10x memory overhead — use it in CI, not production.

7.3 Fixing Races

```
// Fix 1: sync/atomic (fastest for simple counters)
var counter int64
atomic.AddInt64(&counter, 1)

// Fix 2: sync.Mutex (general purpose)
var mu sync.Mutex
var counter int
mu.Lock(); counter++; mu.Unlock()
```

```
// Fix 3: channel (idiomatic for goroutine-to-goroutine handoff)
ch := make(chan int)
go func() { ch <- compute() }()
result := <-ch
```

7.4 The Go Memory Model

The Go memory model defines when one goroutine is guaranteed to observe the effects of writes made by another. The key rules are:

- A send on a channel happens before the corresponding receive completes.
- Closing a channel happens before a receive that returns the zero value.
- sync.Mutex.Unlock() happens before any subsequent Lock().
- sync.WaitGroup.Wait() returns after all Done() calls that balance Add() calls.
- Goroutine creation (go statement) happens before the goroutine starts.

Without these synchronization events, the compiler and CPU are free to reorder memory operations. Always use channels, mutexes, or atomics — never rely on timing or sleep to synchronize.

8. Goroutine Leaks

A goroutine leak occurs when a goroutine is launched but never terminates. It remains blocked forever, holding memory (its stack), any resources it references, and scheduler overhead. Leaks accumulate over time and are a primary cause of memory growth in long-running services.

8.1 Common Causes

Cause	Description
Blocked channel receive	Goroutine waits on <-ch but no sender ever sends a value and the channel is never closed.
Blocked channel send	Goroutine tries ch <- v but no receiver is ready and the buffer is full.
WaitGroup never reaches zero	wg.Add() called more times than wg.Done(), or Done() panics before running.
Infinite loop without exit	Goroutine loops forever with no break condition and no ctx.Done() check.
Context never cancelled	Goroutine waits on ctx.Done() but cancel() is never called (forgot defer cancel()).

```
// GOROUTINE LEAK: blocks on channel receive forever
func leaky() {
    ch := make(chan int)
    go func() {
        val := <-ch // nobody ever sends — goroutine stuck here forever
        fmt.Println(val)
    }()
    // function returns; goroutine is abandoned and never GC'd
}
```

8.2 Detecting Leaks

```
// Runtime: count goroutines
fmt.Println(runtime.NumGoroutine())

// In tests with goleak (go.uber.org/goleak):
func TestMain(m *testing.M) {
    goleak.VerifyTestMain(m)
}

func TestMyFunc(t *testing.T) {
    defer goleak.VerifyNone(t)
    myFunc()
}

// At runtime via pprof:
```

```
// curl http://localhost:6060/debug/pprof/goroutine?debug=2
```

8.3 Preventing Leaks

- Always pass a context to goroutines and check ctx.Done() in loops.
- Ensure channels are closed by the sender when no more values will be sent.
- Use time.After or context.WithTimeout to add timeouts to blocking operations.
- For goroutines waiting in select, include a <-ctx.Done() or <-done case.

```
// CORRECT: goroutine always has a way to exit
func noLeak(ctx context.Context) <-chan int {
    ch := make(chan int)
    go func() {
        defer close(ch)
        for {
            select {
            case ch <- compute():
            case <-ctx.Done():
                return // clean exit on cancellation
            }
        }
    }()
    return ch
}
```

9. Advanced Topics

9.1 runtime.Gosched()

Gosched voluntarily yields the CPU, allowing the scheduler to run other goroutines. Rarely needed since preemptive scheduling (Go 1.14), but can improve fairness in tight compute loops or test code that needs a specific interleaving.

```
for {
    processChunk()
    runtime.Gosched() // yield to allow other goroutines to run
}
```

9.2 runtime.LockOSThread() and UnlockOSThread()

LockOSThread pins the current goroutine to its current OS thread. All goroutine scheduling within that goroutine continues normally, but the goroutine will always run on that specific OS thread. Required when using C libraries, OS APIs, or frameworks that must be called from the same thread (e.g., OpenGL, GUI toolkits, thread-local storage).

```
func openGLInit() {
    runtime.LockOSThread()
    // all OpenGL calls happen here, on a dedicated OS thread
    // the goroutine is pinned; do NOT call defer runtime.UnlockOSThread()
    // unless you are certain all thread-local state is cleaned up
}
```

9.3 Goroutine Stack Growth

Each goroutine starts with a small stack (typically 2–8 KB). When the stack overflows a guard page, the runtime allocates a new, larger stack (usually 2x) and copies all frames to it. The old stack is freed. This process is transparent and can happen many times per goroutine lifetime. The maximum stack size defaults to 1 GB on 64-bit systems (configurable via GOTRACEBACK and debug.SetMaxStack).

- Very deep recursion is safe — no stack overflow unless the 1 GB cap is hit.
- Stack copying invalidates C pointers into Go stack frames — use //go:nosplit or pinned heap allocations for CGO interop.

9.4 GODEBUG and Execution Tracing

```
# Scheduler trace: print scheduler events every 1000ms
GODEBUG=schedtrace=1000 go run main.go
```

```
# GC trace: print GC events
GODEBUG=gctrace=1 go run main.go

# Execution trace (fine-grained goroutine/GC/syscall timeline)
import "runtime/trace"

f, _ := os.Create("trace.out")
trace.Start(f)
defer trace.Stop()
// ... program runs ...

// Analyze: go tool trace trace.out
```

9.5 pprof — Goroutine Profiling

```
import _ "net/http/pprof"

func main() {
    go http.ListenAndServe(":6060", nil) // expose pprof endpoints
    // ...
}

// Dump all goroutine stacks:
// curl http://localhost:6060/debug/pprof/goroutine?debug=2

// Interactive pprof with flamegraph:
// go tool pprof http://localhost:6060/debug/pprof/goroutine

// Count goroutines at runtime:
// curl http://localhost:6060/debug/pprof/goroutine | head -1
```

9.6 Channel Internals (hchan)

Internally, a channel is represented by a runtime structure called hchan containing: a circular buffer (for buffered channels), send and receive queues of waiting goroutines (sudog structs), a mutex for protecting concurrent access, element type information, and a closed flag. Understanding this explains behaviors like why closing a channel wakes all blocked receivers immediately (the runtime walks the receive queue and resumes them all).

10. Testing Concurrent Code

10.1 Always Use -race in CI

```
go test -race ./...
go test -race -count=10 ./... # run multiple times to catch flaky races
go test -race -timeout=120s ./...
```

10.2 Parallel Subtests

```
func TestWorker(t *testing.T) {
    cases := []struct{ name string; input int }{
        {"zero", 0}, {"one", 1}, {"large", 1000},
    }
    for _, tc := range cases {
        tc := tc // capture loop variable (required before Go 1.22)
        t.Run(tc.name, func(t *testing.T) {
            t.Parallel() // run subtests in parallel
            result := worker(tc.input)
            if result < 0 {
                t.Errorf("got %d, want >= 0", result)
            }
        })
    }
}
```

10.3 Detecting Leaks with goleak

```
// go get go.uber.org/goleak

// Option 1: check all tests in package
func TestMain(m *testing.M) {
    goleak.VerifyTestMain(m)
}

// Option 2: per-test check
func TestMyService(t *testing.T) {
    defer goleak.VerifyNone(t) // fails test if any goroutines leak
    svc := NewService()
    svc.Start()
    svc.Stop()
}
```

10.4 Using sync.WaitGroup in Tests

```
func TestConcurrentWrites(t *testing.T) {
```

```
    var counter int64
    var wg sync.WaitGroup
    const goroutines = 500

    for i := 0; i < goroutines; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            atomic.AddInt64(&counter, 1)
        }()
    }
    wg.Wait()

    if counter != goroutines {
        t.Errorf("got %d, want %d", counter, goroutines)
    }
}
```

11. Best Practices and Anti-Patterns

11.1 Best Practices

1. Always give every goroutine a way to exit: context cancellation, a done channel, or channel close.
2. Call wg.Add() before launching the goroutine — never inside it.
3. Always defer wg.Done() and mutex unlocks to guarantee execution on panic.
4. Only the sender closes a channel, and only once.
5. Pass context as the first parameter to any function that starts goroutines or performs I/O.
6. Always defer cancel() after context.WithCancel / WithTimeout / WithDeadline.
7. Use channels to transfer ownership of data; use sync.Mutex to protect shared state.
8. Run go test -race ./... in CI on every commit.
9. Limit concurrency with worker pools or semaphores in services under load.
10. Monitor runtime.NumGoroutine() in production services to catch leaks early.
11. Use errgroup for fan-out operations that can fail.
12. Pass loop variables as function arguments, not by closure capture (for Go < 1.22).

11.2 Anti-Patterns

Anti-Pattern	Problem & Fix
Capturing loop variable in closure	All goroutines see the final value. Fix: pass i as a function argument — go func(n int) { ... }(i).
wg.Add inside the goroutine	Race: Wait may return before Add runs. Fix: always call wg.Add before the go statement.
Forgetting defer cancel()	Context resources leak until parent is cancelled. Fix: always defer cancel() immediately after creating a cancellable context.
Sending on a closed channel	Panics at runtime. Fix: only send from the owner goroutine; close after all sends are done.
Using time.Sleep for synchronization	Fragile, slow, and incorrect under load. Fix: use WaitGroup, channels, or sync.Cond.
Copying sync types (Mutex, WaitGroup)	Copies lose internal state and create independent locks. Fix: always pass by pointer; go vet detects this.
Unbounded goroutine spawning	Exhausts memory under load. Fix: use a worker pool or semaphore.
Goroutine with no exit path	Causes goroutine leaks in long-running services. Fix: always include a ctx.Done() or done channel case.
Ignoring goroutine errors	Errors are silently dropped. Fix: use channels, errgroup, or log from within the goroutine.
Storing context in struct fields	Contexts should flow through call chains, not be stored. Fix: pass ctx as a function parameter.

12. Quick Reference Cheat Sheet

Goroutine Basics

```
go f(args)                              // launch f in new goroutine
go func() { /* body */ }()              // anonymous goroutine
go func(v T) { /* body */ }(val)        // pass by value — safe capture
runtime.NumGoroutine()                  // count live goroutines
runtime.Gosched()                       // yield CPU voluntarily
runtime.GOMAXPROCS(n)                   // set logical processor count
```

Channels

```
ch  := make(chan T)         // unbuffered
ch  := make(chan T, n)      // buffered, capacity n
ch <- v                    // send (blocks if full / no receiver)
v   := <-ch                // receive (blocks if empty)
v, ok := <-ch              // receive + closed check
close(ch)                  // close (sender only, once)
for v := range ch { }      // drain until closed
var r <-chan T = ch         // receive-only alias
var s  chan<- T = ch        // send-only alias
// nil channel: send and receive block forever (useful in select)
```

select

```
select {
case v := <-ch1:           // receive from ch1
case ch2 <- v:             // send to ch2
case <-time.After(d):      // timeout after duration d
case <-ctx.Done():         // context cancelled
default:                   // non-blocking fallthrough
}
```

sync Primitives

```
var mu sync.Mutex
mu.Lock() / mu.Unlock()

var rw sync.RWMutex
rw.RLock() / rw.RUnlock()   // shared read lock
rw.Lock()  / rw.Unlock()    // exclusive write lock

var wg sync.WaitGroup
wg.Add(n) ; wg.Done() ; wg.Wait()
```

```
var once sync.Once
once.Do(func() { /* exactly once */ })

var m sync.Map
m.Store(k,v) ; m.Load(k) ; m.Delete(k) ; m.Range(fn)
```

sync/atomic

```
atomic.AddInt64(&n, delta)
atomic.LoadInt64(&n)
atomic.StoreInt64(&n, v)
atomic.SwapInt64(&n, v)                    // returns old value
atomic.CompareAndSwapInt64(&n, old, new)  // returns bool
var av atomic.Value ; av.Store(v) ; av.Load()
```

context

```
ctx := context.Background()
ctx, cancel := context.WithCancel(parent)
ctx, cancel := context.WithTimeout(parent, duration)
ctx, cancel := context.WithDeadline(parent, time.Time{})
ctx         := context.WithValue(parent, key, val)
defer cancel()           // ALWAYS
<-ctx.Done()             // closed when cancelled / timed out
ctx.Err()                // context.Canceled | context.DeadlineExceeded
```

Testing

```
go test -race ./...                // detect races
go test -race -count=10 ./...      // run 10x for intermittent races
defer goleak.VerifyNone(t)         // detect goroutine leaks
t.Parallel()                       // run subtest in parallel
```

Golden Rule: Prefer channels to communicate between goroutines. Prefer sync.Mutex / atomic to protect state accessed by many goroutines without handoff. Always run with -race. Always give goroutines a way to stop.